Distributed Systems Lab: Programs 2, 3 and 5

Java · Code, how to save, compile and run, and real output

Before you start (read once)

1. **Check Java is installed.** Open a terminal (Command Prompt on Windows) and type `javac -version`. If it prints a version number you are ready. If it says "*not recognized*", install the JDK first.
2. **One folder per program.** For example, make `TCPChat`, `RMI` and `2PC` folders on the Desktop.
3. **Save each file with the exact name shown** (capital letters matter). The file name must match the class name: `public class ChatServer` must be saved as `ChatServer.java`.
4. **Open the terminal inside that folder.**
   - **Windows:** open the folder in File Explorer, click the address bar, type `cmd` and press Enter.
   - **Mac:** open Terminal, type `cd ` (with a space), drag the folder into the window and press Enter.
5. **Always start the server first**, then the client. Each one runs in its **own terminal window**, and all of them are opened in the same folder.

How to save a .java file correctly

- **Notepad (Windows):** paste the code, then **File → Save As**. Choose the folder, set **Save as type: All Files**, and type the name, for example `ChatServer.java`. Without "All Files" it becomes `ChatServer.java.txt` and won't compile.
- **VS Code (Windows / Mac): File → New File**, paste, then **File → Save As** with the exact name.
- **TextEdit (Mac):** first click **Format → Make Plain Text**, then save with the exact name and choose **Use .java**.

Quick overview

Program	Folder	Files to save	Terminals needed	Port
2. TCP Client–Server Chat	`TCPChat`	`ChatServer.java`, `ChatClient.java`	2	5050
3. Java RMI Date and Time	`RMI`	`DateService.java`, `Server.java`, `Client.java`	2	1099
5. Two-Phase Commit (2PC)	`2PC`	`Coordinator.java`, `Participant.java`	3	6789

Program 2: TCP Client–Server Chat Application

Aim: To develop a client–server chat application in Java using TCP sockets.

Theory

- **TCP** (Transmission Control Protocol) is **connection-oriented**: a connection is made first, then data is sent **reliably and in order**.
- The server creates a `ServerSocket` on a port (5050) and calls `accept()`, which waits for a client. The client connects with `new Socket("localhost", 5050)`.
- Messages are sent with `PrintWriter` and received with `BufferedReader`.
- The chat is **turn by turn**: the client sends a message, then the server replies, and so on. Typing `bye` ends the chat.

STEP 1 Save the two files in a folder named `TCPChat`

ChatServer.java

```
import java.io.*;
import java.net.*;

public class ChatServer {
    public static void main(String[] args) throws Exception {
        ServerSocket server = new ServerSocket(5050);
        System.out.println("Server started. Waiting for client...");
        Socket s = server.accept();
        System.out.println("Client connected. Type 'bye' to exit.");

        BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
        BufferedReader kb = new BufferedReader(new InputStreamReader(System.in));

        while (true) {
            String msg = in.readLine();                  // receive from client
            if (msg == null || msg.equalsIgnoreCase("bye")) break;
            System.out.println("Client: " + msg);

            System.out.print("Server: ");
            String reply = kb.readLine();                // type a reply
            out.println(reply);                          // send to client
            if (reply.equalsIgnoreCase("bye")) break;
        }
        System.out.println("Chat ended.");
        s.close();
        server.close();
    }
}
```

ChatClient.java

```
import java.io.*;
import java.net.*;

public class ChatClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 5050);
        System.out.println("Connected to server. Type 'bye' to exit.");

        BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
        BufferedReader kb = new BufferedReader(new InputStreamReader(System.in));

        while (true) {
            System.out.print("Client: ");
            String msg = kb.readLine();                  // type a message
            out.println(msg);                            // send to server
            if (msg.equalsIgnoreCase("bye")) break;

            String reply = in.readLine();                // receive from server
            if (reply == null || reply.equalsIgnoreCase("bye")) break;
            System.out.println("Server: " + reply);
        }
        System.out.println("Chat ended.");
        s.close();
    }
}
```

STEP 2 Compile (once)

Open a terminal in the `TCPChat` folder and type the command below. If nothing is printed, it compiled successfully.

```
javac ChatServer.java ChatClient.java
```

STEP 3 Run

1. **Terminal 1** (server, start this first): `java ChatServer`
2. **Terminal 2** (open another terminal in the same folder): `java ChatClient`
3. Type a message in the **client** window and press Enter. Then type a reply in the **server** window and press Enter. Keep taking turns.
4. Type `bye` in either window to end the chat.

OUTPUT

Yellow text = what was typed on the keyboard.

Terminal 1: Server

```
> java ChatServer
Server started. Waiting for client...
Client connected. Type 'bye' to exit.
Client: Hello Server
Server: Hi Client, how are you?
Client: I am fine. What about you?
Server: I am good too. Nice to chat!
Chat ended.
```

Terminal 2: Client

```
> java ChatClient
Connected to server. Type 'bye' to exit.
Client: Hello Server
Server: Hi Client, how are you?
Client: I am fine. What about you?
Server: I am good too. Nice to chat!
Client: bye
Chat ended.
```

Result: A TCP client–server chat application was developed and executed successfully.

Program 3: Java RMI (Remote Method Invocation): Date and Time Service

Aim: To develop a Java RMI application in which the client gets the current date and time from a remote server.

Theory

- **RMI** lets a Java program **call a method of an object that runs in another JVM**, even on another computer, as if it were a local method.
- **Remote interface** (`DateService`): extends `Remote` and lists the methods the client can call. Each method throws `RemoteException`.
- **Implementation** (`Server`): extends `UnicastRemoteObject` and contains the real code of the method.
- **RMI Registry**: a name service on port **1099**. The server registers its object under a name with `Naming.rebind`, and the client finds it with `Naming.lookup`.
- **Stub**: a client-side proxy created automatically. It sends the method call over the network and brings back the result.

STEP 1 Save the three files in a folder named `RMI`

DateService.java

```
import java.rmi.*;

public interface DateService extends Remote {
    String getDateTime() throws RemoteException;
}
```

Server.java

```
import java.rmi.*;
import java.rmi.registry.*;
import java.rmi.server.*;
import java.util.Date;

public class Server extends UnicastRemoteObject implements DateService {
    Server() throws RemoteException {
        super();
    }

    public String getDateTime() {
        return new Date().toString();
    }

    public static void main(String[] args) throws Exception {
        LocateRegistry.createRegistry(1099);
        Naming.rebind("rmi://localhost/DateService", new Server());
        System.out.println("RMI Server is ready...");
    }
}
```

Client.java

```
import java.rmi.*;

public class Client {
    public static void main(String[] args) throws Exception {
        DateService d = (DateService) Naming.lookup("rmi://localhost/DateService");
        System.out.println("Date and Time from server: " + d.getDateTime());
    }
}
```

STEP 2 Compile (once)

Open a terminal in the `RMI` folder:

```
javac DateService.java Server.java Client.java
```

STEP 3 Run

1. **Terminal 1** (server, start first and leave it open): `java Server`. Wait for "*RMI Server is ready...*".
2. **Terminal 2** (same folder): `java Client`
3. The client prints the server's date and time. Run `java Client` again to get the updated time.
4. To stop the server, press Ctrl+C in Terminal 1.

You don't need to run `rmiregistry` separately, because the server starts it with `LocateRegistry.createRegistry(1099)`.

OUTPUT

Terminal 1: Server

```
> java Server
RMI Server is ready...
```

Terminal 2: Client

```
> java Client
Date and Time from server: Fri Oct 30 21:45:59 IST 2026
```

How it works: The server registers itself in the RMI registry as `DateService`. The client looks up that name and calls `getDateTime()`. The method runs on the server, and the date and time come back to the client.

Result: A Java RMI Date and Time service was developed and executed successfully.

Program 5: Two-Phase Commit (2PC) Protocol Using Coordinator and Participants

Aim: To implement the Two-Phase Commit protocol in Java with one coordinator and two participants using TCP sockets.

Theory

- **2PC** is an **atomic commitment protocol** for distributed transactions: **either all participants commit, or all abort**.
- **Coordinator**: controls the transaction and makes the final decision. **Participants**: vote on whether they are ready.
- **Phase 1, Prepare (voting):** the coordinator sends `PREPARE` to every participant, and each one replies **YES** (ready) or **NO**.
- **Phase 2, Commit / Abort:** if **all** votes are YES, the coordinator sends `COMMIT` to everyone. If **any** vote is NO, it sends `ABORT` to everyone.
- **Drawback:** it is a *blocking* protocol. If the coordinator crashes after PREPARE, the participants have to wait.

STEP 1 Save the two files in a folder named `2PC`

Coordinator.java

```
import java.io.*;
import java.net.*;

public class Coordinator {
    public static void main(String[] args) throws Exception {
        int n = 2;   // number of participants
        ServerSocket server = new ServerSocket(6789);
        System.out.println("Coordinator waiting for " + n + " participants...");

        Socket[] s = new Socket[n];
        BufferedReader[] in = new BufferedReader[n];
        PrintWriter[] out = new PrintWriter[n];

        for (int i = 0; i < n; i++) {
            s[i] = server.accept();
            in[i] = new BufferedReader(new InputStreamReader(s[i].getInputStream()));
            out[i] = new PrintWriter(s[i].getOutputStream(), true);
            System.out.println("Participant " + (i + 1) + " connected");
        }

        // Phase 1: Prepare (voting)
        System.out.println("\nPhase 1: Sending PREPARE to all participants");
        for (int i = 0; i < n; i++) {
            out[i].println("PREPARE");
        }

        boolean allYes = true;
        for (int i = 0; i < n; i++) {
            String vote = in[i].readLine();
            System.out.println("Participant " + (i + 1) + " voted: " + vote);
            if (!"YES".equals(vote)) {
                allYes = false;
            }
        }

        // Phase 2: Commit or Abort
        String decision = allYes ? "COMMIT" : "ABORT";
        System.out.println("\nPhase 2: Decision = " + decision);
        for (int i = 0; i < n; i++) {
            out[i].println(decision);
```

```
            s[i].close();
        }
        server.close();
    }
}
```

Participant.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Participant {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 6789);
        BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
        Scanner sc = new Scanner(System.in);

        System.out.println("Connected to coordinator");
        System.out.println("Coordinator says: " + in.readLine());   // PREPARE

        System.out.print("Ready to commit? (YES/NO): ");
        out.println(sc.nextLine().trim().toUpperCase());              // send vote

        System.out.println("Final decision: " + in.readLine());      // COMMIT or ABORT
        s.close();
    }
}
```

STEP 2 Compile (once)

Open a terminal in the `2PC` folder:

```
javac Coordinator.java Participant.java
```

STEP 3 Run (3 terminals, all in the 2PC folder)

1. **Terminal 1** (coordinator, start first): `java Coordinator`
2. **Terminal 2**: `java Participant`. It waits until the second participant joins. That is normal.
3. **Terminal 3**: `java Participant`
4. Both participants now ask "*Ready to commit? (YES/NO)*". Type **YES** or **NO** in each one and press Enter.
5. **YES** in both gives **COMMIT**. **NO** in either gives **ABORT**. Run all three again to try the other case.

To use 3 participants, change `int n = 2;` to `int n = 3;`, compile again, and open one more terminal running `java Participant`.

OUTPUT

Yellow text = what was typed on the keyboard.

Case 1: both participants vote YES, so the decision is COMMIT

Terminal 1: Coordinator

```
> java Coordinator
Coordinator waiting for 2 participants...
Participant 1 connected
Participant 2 connected

Phase 1: Sending PREPARE to all participants
Participant 1 voted: YES
Participant 2 voted: YES

Phase 2: Decision = COMMIT
```

Terminal 2: Participant 1

```
> java Participant
Connected to coordinator
Coordinator says: PREPARE
Ready to commit? (YES/NO): YES
Final decision: COMMIT
```

Terminal 3: Participant 2

```
> java Participant
Connected to coordinator
Coordinator says: PREPARE
Ready to commit? (YES/NO): YES
Final decision: COMMIT
```

Case 2: one participant votes NO, so the decision is ABORT

Terminal 1: Coordinator

```
> java Coordinator
Coordinator waiting for 2 participants...
Participant 1 connected
Participant 2 connected

Phase 1: Sending PREPARE to all participants
Participant 1 voted: YES
Participant 2 voted: NO

Phase 2: Decision = ABORT
```

Terminal 2: Participant 1

```
> java Participant
Connected to coordinator
Coordinator says: PREPARE
Ready to commit? (YES/NO): YES
Final decision: ABORT
```

Terminal 3: Participant 2

```
> java Participant
Connected to coordinator
Coordinator says: PREPARE
Ready to commit? (YES/NO): NO
Final decision: ABORT
```

Result: The Two-Phase Commit protocol with a coordinator and participants was implemented and executed successfully.

If something goes wrong

Error / problem	Fix
`'javac' is not recognized`	The JDK is not installed or not on PATH. Install the JDK and open a new terminal.
`error: file not found: ChatServer.java`	The terminal is not in the right folder, or the file was saved as `.java.txt`. Save again with **All Files**.
`class X is public, should be declared in a file named X.java`	The file name doesn't match the class name. Rename it exactly, including capital letters.
`Connection refused`	Start the server or coordinator first, then the client or participants.
`Address already in use` / `Port already in use: 1099`	An old server is still running. Close that terminal or press Ctrl + C in it, then try again.
Participant waits and nothing happens	Normal. The coordinator waits until **both** participants have joined.